Тема 1

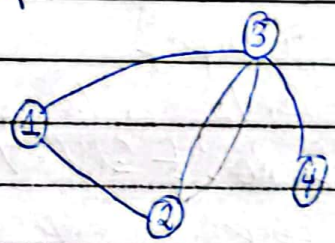
Графи, дървета, обхождане на графи

1. Краен неориентиран мултиграф

Нека V е крайно множество от елементи, които ще наричаме върхове и $E = \{e_1, e_2, \dots, e_m\}$ е крайно множество от елементи, които ще наричаме ребра. Релацията $R_G : E \subseteq V \times V$ дефинираме по следния начин: два върха са в релация, ~~тъй~~ и има ребро между тях. $G(V, E, R_G)$ наричаме неориентиран ~~мулт~~ граф. Ако E е мултимножество ~~от 2 елемента~~, тъст $E \subseteq \{V \times V\}_M$, тогава получаваме мултиграф.

*мултиграф означава, че можем да имаме повече от 1 ребро между два върха от V.

Пример:



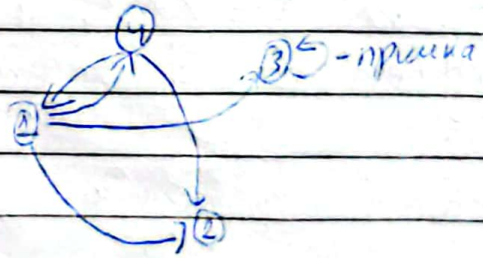
(E - мултимножество от най-с 2 елемента от V) : $f: E \to V^2$

$V^2 = \{(v_1, v_2) \mid v_1, v_2 \in V\}$

2. Ориентиран (мулти)граф

Нека $V = \{v_1, v_2, \dots, v_n\}$ е крайно множество, които ще наричаме върхове и $E = \{e_1, e_2, \dots, e_m\}$ е крайно м-во, които ще наричаме ребра. Функцията $f_G : E \to (V \times V)$, съпоставяща на всяко ребро наредена двойка от върхове, се нарича свързваща функция. $G(V, E, f_G)$ се нарича краен ориентиран (мулти) граф.

Пример:



(E - мултимножество от наредени двойки $f: E \to V \times V$

* при неориентиран граф: ако има ребро между два върха A и B, то можем да стигнем както от A до B, така и от B до A
* при ориентиран граф: ако има ребро от един връх A до друг връх B, то можем да стигнем от A до B през това ребро, но не можем да стигнем от B до A през същото ребро

3. Маршрут (контур) в ориентиран мултиграф
Последователност от върхове $V_1, V_2, \ldots, V_k$ на ориентирания мултиграф $G(V, E, f_G)$ наричаме маршрут в G, ако между всеки два съседни върха от последователността съществува ребро в графа. Формално: $\forall i \in \overline{0,k-1} \mid \exists e \in E: f_G(e) = (V_i, V_{i+1})$. Числото k наричаме дължина на маршрута. Когато началният и крайният връх на маршрута съвпадат, т.е $V_1 = V_k$, тогава маршрута наричаме контур.

4. Път (цикъл) в неориентиран мултиграф
Последователността от върхове $V_1, V_2, \ldots, V_k$ на неориентирания мултиграф $G(V, E, R_G : E \subseteq \{V \times V\}_M)$ наричаме път в G, ако му всеки два съседни върха от последователността съществува ребро в графа. Формално: $\forall i \in \overline{0,k-1} \mid (V_i, V_{i+1}) \in R_G$. Дефинираме тривиалният път от и до и за $\forall u \in V$ да бъде с нулева дължина. Числото k наричаме дължина на пътя. Ако първия и последния връх съвпадат, т.е $V_1 = V_k$, тогава пътят наричаме цикъл.

неориентиран граф (можем да споменем, че в ориентиран граф няма цикъл и обща свързаност)

5. Свързаност и свързани компоненти на граф
Дефинираме релация на свързаност $R \subseteq V \times V$. ~~$\forall x \forall y$~~
За $x, y \in V$ е изпълнено, че $(x, y) \in R \iff \exists$ път от x до y.
R е релация на еквивалентност:
* рефлексивна: $\forall x \mid (x, x) \in R$ (съществува нулев, тривиален път от всеки връх до себе си)

* симетрична: $\forall x \forall y : (x,y) \in R \Rightarrow (y,x) \in R$ (графът е неориентиран $\Rightarrow$ ако съществува път от x до y, то съществува път и от y до x.
* транзитивна: $\forall x \forall y \forall z : (x,y) \in R \wedge (y,z) \in R \Rightarrow (x,z) \in R$ (минавайки през y получаваме път от x до z)

Класовете на еквивалентност на дефинираната релация R определят компонентите на свързаност на графа.
Един граф е свързан, ако ~~класовете~~ релацията R има само 1 клас на еквивалентност (т.е само 1 компонента на свързаност)

6. Дърво и кореново дърво
Деф 1: Дърво наричаме свързан, неориентиран граф, в който няма цикли.

Деф 2: Кореново дърво (индуктивна дефиниция)
База: $T(\{v\}, \emptyset)$ е дърво
Индуктивна стъпка: Нека $T(V,E)$ е дърво и $v \in V$ и $w \notin V$. Тогава $T(V \cup \{w\}, E \cup \{(v,w)\})$ е дърво.

Твърдение: Всяко кореново дърво е дърво
Доказателство:
База: $T(\{v\}, \emptyset)$ е свързан граф без цикли.
Индуктивна хипотеза: ~~Нека~~ Да допуснем, че $T(V,E)$ е свързан граф без цикли.
Индуктивна стъпка:
Добавяме връх $w \notin V$ по следния начин:
$T'(V',E') = T'(V \cup \{w\}, E \cup \{(u,w)\})$
1) От индуктивната хипотеза е ясно, че м/у всеки два върха от V има път, а от w ~~нея~~ съществува път до всеки

друг връх с първо ребро (w,u) Следователно $T'(V \cup \{w\}, E \cup \{(v,w)\})$ е свързан граф.
(графът е бил свързан $\Rightarrow$ v има път от/до всеки друг връх в V, но няма ребро м/у w и V $\Rightarrow$ w е част от същата компонента на свързаност $\Rightarrow$ графът е свързан)
2) Да допуснем, че в T' има цикъл.
Тогава (u,w) ще участва в цикъл, понеже от индукционната хипотеза T е ацикличен и единственият възможен начин да се образува цикъл ще е в следствие на добавяне на новото ребро свързващо новодобавения връх w. Но, за да участва в цикъл w трябва да бъде свързан с поне 2 ребра а w е свързан само с 1 ~~което се противоречи с допускането~~ $\deg(w)=1$
$\Rightarrow$ противоречие. В T' няма цикли.

Твърдение: $|V| = |E| + 1$ за всяко дърво $T(V,E)$
Доказателство:
База: За $T(\{v\}, \emptyset)$ имаме: $|V| = 1$, $|E| = 0$
$\Rightarrow |V| = |E| + 1 = |E| + 1$
Индукционна хипотеза: Допускаме, че за $T(V,E)$ е изпълнено, че $|V| = |E| + 1$.
Индукционна стъпка: Разглеждаме $T'(V', E') = T'(V \cup \{w\}, E \cup \{(v,w)\})$ където $v \in V$, $w \notin V$. Тъй като добавяме нов елемент към дървото
$|V'| = |V| + 1 \Rightarrow |V| = |V'| - 1$
$|E'| = |E| + 1 \Rightarrow |E'| = |E'| - 1$
От индукционната хипотеза имаме, че:
$|V| = |E| + 1 \Rightarrow |V'| - 1 = |E'| - 1 + 1$
$|V'| - 1 = |E'|$
$|V'| = |E'| + 1$ □

7. Покриващо дърво:
Дървото $T(V, E')$ наричаме покриващо дърво за граф $G(V, E)$, ако $E' \subseteq E$.

8. Обхождане на графи

* BFS - обхождане в ширина

Като вход алгоритъма получава граф и начален връх

Нека $G(V, E)$ е графът и x е началният връх.

visited[1, ..., |V|] ← False; // Нека именуване върховете на графа с числата от 1 до |V|. Инициализиране един масив с |V| елемента, където ще пазим дали върхът е посетен. Първоначално маркираме всички върхове като непосетени

queue q; // Инициализиране една опашка. Опашката е структура от данни от тип FIFO - първият влязъл ще бъде първият излязъл от опашката.

q.enqueue(x); // вкарване първоначалния елемент в опашката

visited[x] ← True; // маркиране първоначалния елемент за посетен

while (q is not empty) : // докато има елементи в опашката

v ← q.dequeue(); // взимане първия елемент в опашката

process(v); // извършване някаква бизнес логика с елемента

foreach(neighbour n of v): // обхождане всички деца на върха v

if (visited[n] == False); // ако не е било посетено детето

visited[n] ← False; // маркиране го като посетен

q.enqueue(n); // добавяне го в опашката

* DFS - обхождане в ~~дълбочина~~ дълбочина

Получава граф $G(V, E)$ и първоначален връх x като вход

visited[1, ..., |V|] ← False;

stack s; // Инициализиране празен стек. Стек е структура

от данни от тип LIFO. Последният влязъл, ще бъде първи излязал

```
S.push(x); // Добавяне първоначалния елемент към стека
~~visited[x] = true; // Маркиране първия елемент за посетен~~
while (S is not empty); // Докато има елементи в стека
    v ← S.pop(); // Взимане последния елемент в стека
    process(v); // Обработване върха v
    visited[v] ← true;
    foreach (child c of v): // Обхождане всички деца на v
        if (visited[c] == false) // Проверяване дали вече е
            било обходено това дете на v
            ~~visited~~ S.push(c); // Добавяне го в стека
```

- Може да се имплементира и чрез рекурсия

8. Ойлеров път и Ойлеров цикъл

Път в свързания граф G, който минава през всяко ребро в G точно веднъж, наричаме Ойлеров път. Ако първият и последният връх на Ойлеровия път съвпадат, то казваме, че графът е Ойлеров.

Нека $G(V, E)$ е неориентиран мултиграф.

Броят на ребрата, които излизат от даден връх $v \in V$, наричаме степен на връх v и бележим $d(v)$

Един граф G е Ойлеров (т.е има Ойлеров цикъл) $\Leftrightarrow$ G е свързан и всеки негов връх е от четна степен.

Един граф G има Ойлеров път, който не е цикъл $\Leftrightarrow$ G е свързан и има точно два върха от нечетна степен

Изпит теория с доказателства по ДСТР 1

13.02.2021

Атанас Василев, 62577, 6-та група

зад1. Формулирайте и докажете теорема на Ойлер.

Нека $G(V, E)$ е свързан граф. Тогава G е Ойлеров т.т.к всеки връх на G има четна степен.

Доказателство:

1) Нека ребрата на $G(V, E)$ образуват Ойлеров цикъл. Тогава всеки връх има четна степен, защото при обхождането на графа по Ойлеровия цикъл, на всяко ребро „влизащо" във върха v_i съответства ребро (различно от „влизащото" според дефиницията за път), „излизащо от v_i", а цикълът съдържа всички ребра точно по един път.

2) Нека $G(V, E)$ е свързан и такъв, че всеки връх има четна степен. Ще опишем алгоритъм, който построява Ойлеров цикъл от всички ребра на графа.

а) Нека r е произволен връх на графа. Обявяваме го за текущ t.

б) Докато е възможно строим път по следното правило: от върха t, в който се намираме, преминаваме в някой съсед t по необходимо ребро. Използваното ребро обявяваме за обходено, а върхът t за текущ. Да допуснем, че когато правилото от б) не е вече приложимо то $t \neq r$. Тогава ще се окаже, че степента на t е нечетна.

Понеже възлайки за последен път в t
сме отбелязали като обходено едно ребро, а при
всяко следващо минаване през този връх сме отбелязвали
като обходени по две негови ребра. Това е
противоречие. Следователно, когато правилото от
стъпка б) не е вече приложимо, t=r всички
обходени на този етап ребра образуват цикъл C.
в) Ако цикълът C съдържа всички ребра ~~кра~~ приключваме обхождането. Построен е
Ойлеров цикъл.
г) В противен случай премахваме от графа
ребрата на намерения цикъл. Получения граф
е такъв, че всеки връх е с четна степен.
Търсим от построения цикъл връх r', от който
излиза поне едно необходено ребро.
Ако допуснем, че такъв връх няма, при
условието, че не всички ребра са обходени, ще получим
противоречие със свързаността на графа. Повтаряме
действията от стъпка б) с начален връх r=r' и
построяваме нов цикъл. Сега разглеждаме двата
цикъла в общия им връх r' и с отъждествява-
-нето на получените краища два по два,
получаваме нов цикъл C и преминаваме към в)

Алгоритъмът ще завърши работа, когато всички
ребра се изчерпят и построява Ойлеров цикъл.
Следователно графа е Ойлеров и ☺ теоремата
е доказана.